

Assignment 1

Course: B.Tech. CSE

Subject: System Design

Submission Date: 07-Septemeber-2026

Semester: 5

Subject Code: TCS-504

1. The problem

Build a small **movie ticket booking system** for a single cinema (like PVR or INOX), as a menu-driven C++ console program.

A customer should be able to see which movies are playing, pick a show, see which seats are free, book seats, pay, get a ticket, and cancel a booking.

2. Scope — build exactly this, nothing more

Features you MUST build (8)

#	Feature
---	---------

F1	List all movies currently playing
----	-----------------------------------

F2	For a chosen movie, list its shows (screen + start time)
----	--

F3	For a chosen show, display the seat layout with AVAILABLE / BOOKED status
----	--

F4	Book one or more seats for a show (reject an already-booked seat)
----	---

F5	Price the booking by seat type : SILVER ₹150, GOLD ₹250, PLATINUM ₹400
----	---

F6	Pay by UPI, Card or Cash — a failed payment must NOT confirm the booking
----	---

F7	Print a ticket: booking id, movie, screen, time, seat numbers, total amount
----	---

F8	Cancel a booking — the seats become AVAILABLE again
----	---

3. What you must submit (7 items)

Item	What it is
A	Requirement analysis (FR + NFR)
B	Noun–verb analysis table (list of classes and their methods)
C	Relationship table with justification
D	Class diagram
E	Sequence diagram for "book a ticket and make payment"
F	Modular working code + demo run
G	SOLID mapping + one thing you deliberately did NOT do

4. Step A — Requirement analysis

Write **functional requirements** (FR) precisely enough to be testable, and 3–4 **non-functional requirements** (NFR).

Two FRs are done for you as the standard to match:

FR4 — Booking: a customer selects one or more seat numbers for a show. If **any** selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed **only after** payment succeeds.

FR6 — Payment: exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.

Now write **FR1, FR2, FR3, FR5, FR7, FR8** yourself in the same style.

Hint for NFRs: think about modularity (one class per file), extensibility (adding NetBanking must not force edits to existing classes), and validating bad input.

5. Step B — Noun–verb analysis

Underline the nouns and verbs in the problem statement. Then fill this table. **Rejecting a noun is also design** — say why.

Noun found	Keep as a class?	Reason
Movie	yes	has its own data and identity
Seat	yes	has number, type, price
"seat layout"	no	it is a <i>view</i> of a Show's seats, not a thing → a print method
... complete the rest ...		

6. Step C — The classes you need

These are the classes required. **The list is given so nobody gets stuck** — your job is to decide each class's **data members**, **methods and access modifiers**, and to fill the "must NOT do" column.

Core entity classes

Class	its ONE responsibility
Movie	title, language, duration — nothing else
Seat	one physical seat: number and type (SILVER/GOLD/PLATINUM)
Screen	one auditorium: screen number; owns its seats
Cinema	the theatre: name; owns its screens
Show	one screening = a Movie on a Screen at a time; owns its ShowSeats
ShowSeat	the status of ONE seat FOR ONE show (AVAILABLE / BOOKED)
Customer	name and phone

Class	Its ONE responsibility
-------	------------------------

Booking booking id, which show, which seats, total amount, status

Behaviour / service classes

Class	Its ONE responsibility
-------	------------------------

Payment (*abstract*) the payment contract: pay(amount) — pure virtual

UpiPayment, CardPayment, CashPayment how each method actually pays

PriceCalculator turn a list of seats into a total amount

TicketPrinter format and print a ticket — **printing only**

BookingService the orchestrator: runs the booking flow end to end

Cinema/main menu the console menu and input reading

Why ShowSeat and not just Seat? Seat A1 exists once in the screen, but its *status* is different for every show. A1 may be booked for the 6 PM show and free for the 9 PM show. Status belongs to the show, not to the physical chair. Getting this right is worth real marks.

For each class also state, in one line each: what it **knows**, what it **does**, and what it **must NOT do**.

7. Step D — Relationships

For **every** pair below, choose composition / aggregation / association / inheritance **and justify it with the lifetime test: if the whole is destroyed, does the part die?**

Pair	Your choice	Your justification
------	-------------	--------------------

Cinema — Screen ?

Screen — Seat ?

Show — Movie ?

Show — Screen ?

Pair	Your choice	Your justification
Show — ShowSeat	?	
Booking — Customer	?	
Booking — ShowSeat	?	
Booking — Payment	?	
Payment — UpiPayment	?	
BookingService — Booking	?	

One is worked out for you as the standard:

Show — Movie = AGGREGATION (◇). A Show *borrow*s a Movie. Cancel the 6 PM show and the movie "3 Idiots" still exists and still plays at 9 PM. The Show stores a pointer/reference and never deletes it.

Warning: answers with no justification score zero for that row. "Because it feels like has-a" is not a justification. Use the lifetime test.

8. Step E — Class diagram

Draw **one** class diagram containing every class from Step C.

Your diagram must show:

- three compartments per class: **name / attributes / methods**
- visibility markers: - private, # protected, + public
- correct notation: ◆ filled diamond = composition, ◇ hollow diamond = aggregation, —▶ open arrow = association, —▷ hollow triangle = inheritance
- **multiplicities** — e.g. Screen 1 ◆— 1..* Seat, Show 1 ◇— 1 Movie
- the Payment hierarchy with «abstract» marked on the base

Hand-drawn on paper is fine if it is neat and photographed clearly. Tools you may use: draw.io

9. Step F — Sequence diagram

Draw the sequence diagram for **ONE use case: "customer books 1 seat and pays by UPI"**, from the customer choosing a show to the ticket being printed.

Your diagram must contain, and be labelled with:

- **lifelines** for at least: customer, bookingService, show, showSeat, priceCalculator, payment, booking, ticketPrinter
 - **activation bars** showing when each object is busy
 - **synchronous messages** for ordinary method calls, For example: Object creation, etc.
 - **return messages** for values coming back
 - **«create»** messages where an object is created (Booking, Payment)
-

10. Step G — Modular code

Course rules

- **One class per file.**
- **No header files**

Suggested file structure

- **File Structure in case of C++**

01_Movie.cpp	06_ShowSeat.cpp	11_PriceCalculator.cpp
02_Seat.cpp	07_Customer.cpp	12_TicketPrinter.cpp
03_Screen.cpp	08_Booking.cpp	13_BookingService.cpp
04_Cinema.cpp	09_Payment.cpp	main.cpp
05_Show.cpp	10_PaymentTypes.cpp	

- **File Structure in case of Java**

01_Movie.java	06_ShowSeat.java	11_PriceCalculator.java
02_Seat.java	07_Customer.java	12_TicketPrinter.java
03_Screen.java	08_Booking.java	13_BookingService.java
04_Cinema.java	09_Payment.java	main.java
05_Show.java	10_PaymentTypes.java	

OOP concepts that MUST appear (and be pointed out in a comment)

OOP Concept	C++ Implementation Hint	Java Implementation Hint
Encapsulation	Keep seatStatus and bookingAmount private and modify them only through validation methods.	Declare variables as private and provide controlled access through getters, setters, or dedicated methods such as bookSeat() and cancelSeat().
Abstraction	Create an abstract Payment class with a pure virtual method virtual bool pay(double amount) = 0;.	Create an abstract class Payment with an abstract method abstract boolean pay(double amount);.
Inheritance	UpiPayment, CardPayment, and CashPayment inherit from Payment.	Use extends to derive UpiPayment, CardPayment, and CashPayment from the Payment class.
Runtime Polymorphism	Payment* p; p->pay(total); invokes different implementations at runtime.	Payment p = new UpiPayment(); p.pay(total); demonstrates dynamic method dispatch.
Compile-Time Polymorphism	Use overloaded constructors or overloaded methods.	Create multiple constructors or methods with different parameter lists.
Static Members	Use a static variable to generate unique booking IDs.	Use a static variable such as static int nextBookingId; that is shared by all objects.
this Keyword	Use this inside constructors and methods to refer to the current object.	Use this to distinguish instance variables from local variables and to invoke another constructor using this().
Composition	Cinema → Screen, Screen → Seat, Show → ShowSeat	Create objects inside other classes. For example, Cinema contains a collection of Screen objects. If the parent object is destroyed, the child objects no longer exist.
Aggregation	Show → Movie, Booking → ShowSeat	Store references to existing objects instead of creating them. The child object can exist independently of the parent object.
Association	Customer → BookingService	Customer interacts with BookingService to perform operations, but neither object owns the other.

Edge cases your demo must handle

1. booking a seat that is already BOOKED → rejected, nothing changes
 2. a failed payment → booking NOT confirmed, seats released
 3. cancelling a booking → those seats show AVAILABLE again
 4. an invalid seat number or menu choice → a clear message, no crash
-

11. Step H — SOLID

Apply **at least three** of the five principles, and name where.

Principle	A hint for this system
S	should <code>Booking</code> also print the ticket? Who asks for a layout change vs a pricing change?
O	adding NetBanking must need a new class only — no edit to <code>BookingService</code>
L	every <code>Payment</code> child must work through a <code>Payment*</code> with no extra setup call
I	if only some payments support refunds, should <code>Payment</code> force <code>refund()</code> on all?
D	should <code>BookingService</code> create a <code>CardPayment</code> itself, or receive a <code>Payment&</code> ?

12. Clean code checklist

- [] intention-revealing names — `bookedSeatCount`, not `bsc` or `c1`
 - [] no number-series names — no `list1`, `list2`, `temp1`
 - [] every function does ONE thing; if you need "and" to describe it, split it
 - [] no function longer than ~20 lines
 - [] maximum 2 levels of indentation; name your conditions (`bool isSeatFree = ...; if (isSeatFree && isPaid)`)
 - [] 0–2 parameters preferred; 4+ means they want to become a struct/class
 - [] no side effects — a function named `showSeats()` must not book anything
 - [] constants instead of magic numbers — `SILVER_PRICE`, not `150`
 - [] no repeated code (DRY)
-

13. Expected demo output (roughly this shape)

===== MOVIE TICKET BOOKING =====

1. Movies 2. Book 3. Cancel 4. My tickets 0. Exit

Choose: 1

[1] 3 Idiots Hindi 170 min

[2] Interstellar English 169 min

Choose movie: 1

[1] Screen-1 06:00 PM

[2] Screen-2 09:00 PM

Choose show: 1

SCREEN-1 06:00 PM | 3 Idiots

SILVER A1[] A2[X] A3[] A4[]

GOLD B1[] B2[] B3[X]

PLATINUM C1[] C2[]

([] = available [X] = booked)

Seats (e.g. A1,B2): A1,B2

A1 SILVER Rs.150

B2 GOLD Rs.250

TOTAL Rs.400

Pay by: 1.UPI 2.Card 3.Cash > 1

[UPI] Rs.400 paid successfully

===== TICKET =====

Booking ID : BK1001

Movie : 3 Idiots

Screen : Screen-1 06:00 PM

Seats : A1, B2

Amount : Rs.400 Status: CONFIRMED

=====